

Executive Summary

We propose a **Phase 1 MVP** architecture for TeamOS that includes Authentication, Organization/Team management, Chat, Tasks, and Documents. The backend will use **Java Spring Boot** (microservices or modular-monolith style) and the frontend will use **Next.js (React + TypeScript)**. Spring Boot is well-suited for Java microservices – “start small and iterate fast” – and is a de facto standard for production services [8†L28-L34] . Next.js is a React framework for full-stack apps that automates bundling and optimizes performance [6†L333-L340] . We'll assume **AWS-like cloud** (AWS RDS for Postgres, S3-compatible for R2, ElastiCache Redis, etc.) unless otherwise specified.

Our **deliverables** will include scaffolded code and configs: Spring Boot services (controllers, services, repositories), SQL migrations for PostgreSQL, OpenAPI/Swagger specs, a Next.js app (pages/components and auth flows), WebSocket/STOMP chat, rich-text/Markdown editors, CI/CD (GitHub Actions), Dockerfiles, Docker Compose or Kubernetes YAML, Terraform templates for infrastructure, and unit/integration tests. Non-functional needs include JWT auth (with refresh tokens), RBAC roles per org/team, multi-tenant data isolation, logging/metrics (observability), and security best practices (e.g. JWT validation [27†L261-L267]).

To manage scope, we will break the work into **milestones** (project setup, auth, org/teams, chat, tasks, docs, final polish) with clear acceptance criteria. We will also include a timeline chart (Gantt or Mermaid), and tables comparing architecture choices (monolith vs microservices) and DB multi-tenancy models. This prompt text instructs the code-generation model to generate a complete Git-like project layout (with a top-level README) implementing the above. Throughout, we refer to official docs (Spring Boot, Next.js, PostgreSQL, Redis, Cloudflare R2, STOMP, OAuth2/JWT) to ground our decisions [8†L28-L34] [6†L333-L340] [15†L169-L173] [13†L64-L72] [17†L259-L263] [27†L261-L267] .

Requirements and Scope

Based on the PRD/Design/TRD, **TeamOS Phase 1** will deliver:

- **Authentication:** User signup, login, JWT access+refresh tokens (optionally OAuth2 flows).
- **Multi-Tenant Organizations & Teams:** Create organizations (tenants) and teams within them; assign users with roles (e.g. admin/member) [27†L261-L267] . Enforce data isolation per organization.
- **Chat:** Real-time messaging (channels/threads) across teams and orgs. Web UI to send/receive messages (via WebSockets/STOMP).
- **Tasks:** CRUD task management (to-do lists), assignable to users/teams.
- **Documents:** Create and edit documents (rich-text or Markdown). Versioned docs within teams/orgs.

Each organization's data must be isolated. We will use a single PostgreSQL database with a `tenant_id` column (a “pool” model) for agility [11†L51-L59] . This is simpler to manage than a separate database per org (“silo”), which offers stronger isolation but higher overhead [11†L33-L41] [11†L51-L59] . RBAC will enforce that users see only their org's data. Observability (logging, metrics via Micrometer/Spring Actuator) and security best practices (strong password hashing, HTTPS, input validation) will be built in.

Architecture and Service Boundaries

For Phase 1 we recommend a **modular monolith** or a small set of services (4–6) rather than dozens of microservices, to reduce early complexity [25†L68-L72] . Possible service boundaries are:

- **Auth Service:** Handles user registration/login, JWT issuance/refresh, and OAuth2 connectors. (Also stores user profiles, roles, password hashes.)
- **Org/Team Service:** Manages organizations and teams, membership, and RBAC rules.
- **Chat Service:** Manages real-time messaging; WebSocket/STOMP endpoints for pub-sub chat.
- **Task Service:** CRUD APIs for tasks, including assignees and status.
- **Document Service:** CRUD for documents, with content (e.g. Markdown) and metadata.
- **API Gateway (optional):** A single entry point (Spring Cloud Gateway or Spring Boot with Zuul/Nginx) routing requests to services and handling auth tokens.

Each service would use **Spring Boot** with Spring Web and Spring Data JPA. We'll use **PostgreSQL** for all persistent data, **Redis** for caching and as a STOMP message broker, and **Cloudflare R2** (or AWS S3) for storing uploaded files/attachments. WebSockets with STOMP messaging will power the chat (Spring's `@EnableWebSocketMessageBroker` config) [17†L259-L263] . OpenAPI (via Springdoc) will auto-generate REST API docs.

The architecture trade-offs are summarized below: a modular monolith has lower initial overhead, whereas full microservices give per-service scaling. Starting as one codebase is easier to develop [25†L68-L72] , and you can split modules later.

Criterion	Monolithic/Modular Monolith	Microservices
Deployment	Single deployable unit [25†L58-L64] ; simple CI/CD.	Many deployables; complex CI/CD.
Complexity	Lower operational complexity to start [25†L68-L72] .	Distributed complexity: network, auth, messaging.
Scalability	Scales all components together.	Scale each component independently.
Data Management	Shared database; simpler queries.	Separate DBs or schemas; eventual consistency.
Fault Isolation	One service failure can impact all.	Failures are isolated to that service.
Team Autonomy	Tighter coupling between teams.	Teams work in isolation per service.

Implementation Tasks

Backend (Spring Boot)

- **Project Setup:** Initialize one (or multiple) Spring Boot projects (Java 17+). If monolithic, use *Spring Modulith* or multi-module Maven/Gradle to organize domains. Otherwise set up separate services.

Include Spring Security, Spring Web, Spring Data JPA, and springdoc-openapi. Enable WebSocket Message Broker (`@EnableWebSocketMessageBroker` 【17†L259-L263】).

- **Entities & Repositories:** Define JPA entities/tables for **Users** (`id`, `email`, `password_hash`, `tenant_id`, `role`), **Organizations** (`id`, `name`, `owner_id`), **Teams** (`id`, `name`, `org_id`), **Messages** (`id`, `team_id`, `sender_id`, `content`, `timestamp`), **Tasks** (`id`, `title`, `description`, `assignee_id`, `team_id`, `status`, `due_date`), **Documents** (`id`, `title`, `content`, `owner_id`, `team_id`, `version`). Include indexes on `tenant_id` and foreign keys. Provide Flyway (or Liquibase) SQL migrations that create these tables.
- **Security (Auth):** Implement user signup/login endpoints. On login, return a signed JWT access token (e.g. with JWT) and a refresh token. Use **short-lived JWTs** with strong signature (as per best practices 【27†L261-L267】). Store refresh tokens in DB or use stateless JWT rotation. Add OAuth2 login placeholders (e.g. Google) if time permits.
- **RBAC:** In each service, check JWT claims to enforce that every request includes a `tenant_id` (org) and that the user's role in that org allows the action. For example, only org admins can create teams or invite users.
- **Organization/Team APIs:** CRUD endpoints for organizations and teams. Endpoints like `POST /api/orgs`, `GET /api/orgs/{orgId}`, `POST /api/orgs/{orgId}/teams`, etc., with request/response JSON. Ensure when creating a user/org, the `tenant_id` is set and queries always filter by `tenant_id`. (E.g. repository methods use `WHERE tenant_id = :currentTenant`.)
- **Chat (WebSocket):** Configure a STOMP endpoint (e.g. `/ws`) and message broker (simple or use Redis pub/sub). Define topics per team (e.g. `/topic/team.{teamId}`). Example message JSON: `{"teamId":123,"senderId":45,"content":"Hello","timestamp":...}`. Implement controllers so that sending to `/app/chat.sendMessage` broadcasts to `/topic/team.{teamId}`.
- **Task APIs:** REST endpoints under `/api/tasks`. Example: `POST /api/tasks` with body `{"title":"Fix bug","teamId":123,...}` returns the created task JSON. Support `GET /api/tasks?teamId=123` to list. Link tasks to teams/users by IDs.
- **Document APIs:** Similar CRUD endpoints `/api/docs`. Documents may be simple markdown text. Example: `POST /api/docs` with `{"title":"Spec","content":"**Design**...", "teamId":123}`. Store content in DB or in R2 as files.
- **Real-Time Notifications (Optional):** When chat or tasks change, publish events (could be implemented as part of WebSockets or via Redis).
- **OpenAPI/Swagger:** Use springdoc-openapi to generate Swagger UI for all REST endpoints. Annotate controllers with `@Operation` comments and example schemas.
- **Configuration:** Use application properties for DB (JDBC URL, credentials), Redis, Cloud R2 credentials, and JWT secrets. Support multiple profiles (dev, prod).

Throughout the code, **comment** each major class and method. For example, in controllers: `// Endpoint for user signup`, and Javadoc for services. Include a `README.md` in each module describing its purpose and build/run steps.

Frontend (Next.js)

- **App Setup:** Create a Next.js 13+ project with TypeScript. Implement a **monorepo** or separate repos? (Likely one Next.js app). Include ESLint/Prettier config, React context for auth, and routing.

- **Authentication Pages:** Pages for **Login** and **Signup** (e.g. `/login.tsx`, `/signup.tsx`). Forms posting to backend APIs (`/auth/login`, `/auth/signup`). Store JWT (in memory or HttpOnly cookie) and attach to API calls.
- **Auth Flow:** After login, redirect to a protected dashboard. Implement a higher-order component or hook to check auth token and role. Handle token refresh (call `/auth/refresh`).
- **Organization/Team UI:** A dashboard page where a user (within one org) can view org info and create teams. E.g. `/orgs/[orgId]/teams`. Forms to invite users (POST `/orgs/{orgId}/teams/{teamId}/members`).
- **Chat UI:** A team chat page (e.g. `/orgs/[orgId]/teams/[teamId]/chat`). Connect to the STOMP WebSocket (e.g. using `@stomp/stompjs` or native WebSocket). Example: subscribe to `/topic/team.{teamId}` and display messages. Provide an input to send messages (publish to `/app/chat.sendMessage`).
- **Task UI:** Pages for task lists (`/orgs/[orgId]/teams/[teamId]/tasks`) and details. Use React state or hooks to fetch tasks from the API. Forms to create/edit tasks.
- **Document Editor:** A page for document editing (`/orgs/[orgId]/teams/[teamId]/docs/new` or `[docId]/edit`). Integrate a rich text editor (e.g. React Markdown editor like [react-markdown](#) or [Slate](#)). Allow saving to backend.
- **Layout & Components:** Common UI (header with nav, org selector, user menu). Use CSS framework or component library (e.g. Tailwind CSS or Material-UI).
- **API Integration:** Create utility (e.g. Axios instance) to call backend with JWT. Add error handling (401 redirect to login).
- **Testing:** Write basic tests with Jest/React Testing Library for key components (login form, task list).

DevOps and Infrastructure

- **Docker:** Write `Dockerfile` for each backend service (using Maven builder image, etc.) and for Next.js (node build). Also a `docker-compose.yml` to run Postgres, Redis, and all services locally.
- **Kubernetes:** Provide sample K8s manifests (Deployment + Service) for each service and frontend. E.g., a `deployment.yaml` for `auth-service`, `org-service`, etc., pointing to Docker images. Use `ConfigMap` / `Secret` for env vars (DB URL, JWT secret).
- **Terraform:** Write Terraform scripts to provision cloud resources on AWS:
 - A **PostgreSQL** instance (RDS) with appropriate subnet and security group.
 - A **Redis** cluster or ElastiCache instance.
 - A **Cloudflare R2** bucket or S3 bucket (via an AWS S3-compatible provider) for file storage.
- (Optional) A VPC and subnets for isolation.
 - Use Terraform AWS provider (or Cloudflare provider for R2). Example: create an `aws_db_instance` for Postgres.
- **CI/CD (GitHub Actions):**
 - A workflow for **backend**: on push, run `mvn test`, build Docker image, push to registry.
 - A workflow for **frontend**: run `npm test`, `npm build`, push image.
 - (Optional) deploy to Kubernetes after push.
 - Show example YAML steps (checkout, set up JDK/Node, build, test, docker).

- **Observability:** Include logging (SLF4J) and metrics (Micrometer with Prometheus exporter). Provide a metrics endpoint (Spring Actuator).
- **Security Best Practices:** Use HTTPS (certificate management out-of-scope but note it). Sanitize inputs to avoid injections. Rate-limit login attempts (Spring Security features).
- **Output README:** The generated code should include a top-level `README.md` with instructions to set up the database, run containers, and run the app. Mention required environment variables (DB URLs, JWT secret, R2 keys).

Below is a sample **repository structure** to organize the output:

```
/backend
  /auth-service      // Spring Boot app (Auth/User management)
  /org-service       // Spring Boot app (Orgs & Teams)
  /chat-service      // Spring Boot app (Chat and WebSockets)
  /task-service      // Spring Boot app (Tasks)
  /doc-service       // Spring Boot app (Documents)
  /api-gateway       // (Optional) Spring Cloud Gateway
  Dockerfile         // parent Dockerfile for all or per-service
  docker-compose.yml // development compose file (Postgres, Redis, etc.)
  .github/workflows  // GitHub Actions YAML
  terraform/         // Terraform templates (main.tf, variables.tf)
  k8s/               // Kubernetes manifests (YAMLs)
  README.md
/frontend
  /pages             // Next.js pages (e.g. index.tsx, login.tsx, org/[id]/
teams)
  /components        // React components (ChatWindow, TaskList, etc.)
  /lib               // helpers (API client, auth context)
  Dockerfile
  next.config.js
  tsconfig.json
  .github/workflows
  README.md
```

API & Schema Examples

- **Sample Endpoint (Auth):**

```
POST /api/auth/signup
Request JSON: { "email": "user@example.com", "password": "Passw0rd!" }
Response JSON: { "userId": 123, "token": "<jwt>", "refreshToken":
"<token>" }
```

- **Sample DB Table (users):**

```

CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(50) NOT NULL,          -- e.g. "ADMIN", "MEMBER"
  tenant_id INT NOT NULL,
  FOREIGN KEY (tenant_id) REFERENCES organizations(id)
);
CREATE INDEX idx_users_tenant ON users(tenant_id);

```

• WebSocket Message (Chat):

```

{
  "teamId": 12,
  "senderId": 45,
  "senderName": "Alice",
  "content": "Hello team!",
  "timestamp": "2026-08-04T12:34:56Z"
}

```

• Next.js Page Example:

```

// pages/login.tsx
import { useState } from 'react';
import { useRouter } from 'next/router';
function LoginPage() {
  const [email, setEmail] = useState(''), [pw, setPw] = useState('');
  const router = useRouter();
  async function handleLogin() {
    const res = await fetch('/api/auth/login', { method: 'POST', body:
JSON.stringify({email, password: pw}) });
    if (res.ok) { router.push('/dashboard'); }
  }
  return (
    <div>
      <h1>Login</h1>
      <input value={email} onChange={e => setEmail(e.target.value)}
placeholder="Email"/>
      <input type="password" value={pw} onChange={e =>
setPw(e.target.value)} placeholder="Password"/>
      <button onClick={handleLogin}>Sign In</button>
    </div>
  );
}
export default LoginPage;

```

• **Dockerfile (example for Spring Boot):**

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY mvnw pom.xml ./
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B
COPY src src
RUN ./mvnw package -DskipTests
ENTRYPOINT ["java", "-jar", "target/auth-service.jar"]
```

• **GitHub Actions (Java build):**

```
name: Backend CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
      - name: Build and test
        run: mvn test package -B
      - name: Build Docker image
        run: docker build -t teamos/auth-service:latest ./backend/auth-
service
      - name: Publish image
        run: docker push teamos/auth-service:latest
```

• **Terraform Snippet (AWS RDS):**

```
resource "aws_db_instance" "teamdb" {
  allocated_storage = 20
  engine            = "postgres"
  engine_version    = "15"
  instance_class    = "db.t4g.micro"
  name              = "teamosdb"
  username          = var.db_user
  password          = var.db_pass
  skip_final_snapshot = true
}
resource "aws_s3_bucket" "teamos_r2_bucket" {
```

```
bucket = "teamos-storage"
}
```

Libraries, Frameworks, and Tools

Key technologies include:

- **Spring Boot + Spring Security:** For backend microservices. (See Spring docs on microservices [8†L28-L34] .)
- **Spring Data JPA:** ORM to PostgreSQL.
- **Spring Web & WebSocket (STOMP):** For REST APIs and real-time messaging [17†L259-L263] .
- **springdoc-openapi:** Auto-generate Swagger UI.
- **Next.js (React + TS):** Frontend framework [6†L333-L340] .
- **Redis:** In-memory store for caching and also as a message broker for STOMP (publish-subscribe). Redis supports streams and pub/sub for real-time workloads [13†L64-L72] .
- **PostgreSQL:** Relational DB for user/org/task data.
- **Cloudflare R2 (S3):** Object storage for uploaded files and document assets. R2 is S3-compatible and offers “no egress fees” [15†L169-L173] .
- **CI/CD:** GitHub Actions; Docker/Docker Compose; Kubernetes (YAML manifests).
- **Terraform:** AWS provider for RDS, ElastiCache, S3/R2 buckets.
- **Authentication Libraries:** For JWT signing (e.g. `io.jsonwebtoken:jjwt`), and optionally Spring Authorization Server for OAuth2.
- **Testing:** JUnit 5 for backend, Jest/React Testing Library for frontend.

We will follow best practices from each domain: e.g. Spring’s guide for WebSocket chat [17†L259-L263] , Next.js official docs (TypeScript support, routing) [6†L333-L340] , JWT/OAuth2 patterns [27†L261-L267] , and AWS guidance on multi-tenant databases [11†L51-L59] .

Non-Functional Requirements

- **Authentication & Security:** Use JWT access tokens (short-lived) plus refresh tokens [27†L261-L267] . Validate token signature, issuer, and audience on every request [27†L261-L267] . Implement RBAC so users can only act within their org/team. Consider CSRF protection if using cookies. Follow OWASP guidelines (sanitize all inputs, use HTTPS).
- **Multi-Tenancy:** Each organization is a tenant. We will use a **“pool” model** (shared schema, tenant_id in tables) for simplicity [11†L51-L59] . This allows centralized management and is more agile, at the cost of stricter query filtering and some compliance risk. (The alternative “silo” model with one DB per tenant has no cross-tenant impact but much more overhead [11†L33-L41] .)
- **Performance & Scalability:** Design stateless services so they can scale horizontally. Use connection pooling for DB. Cache frequent reads in Redis. The microservices can be deployed behind a load balancer or scaled up as needed.
- **Logging & Observability:** Use structured logging (SLF4J) and expose metrics (via Micrometer). Implement health checks (`/actuator/health`). Monitor request latencies and error rates.
- **Testing & Quality:** Write unit tests for all layers. Strive for $\geq 80\%$ coverage. Use integration tests (e.g. SpringBootTest with Testcontainers for PostgreSQL) to validate endpoints. On the frontend, include snapshot/unit tests.

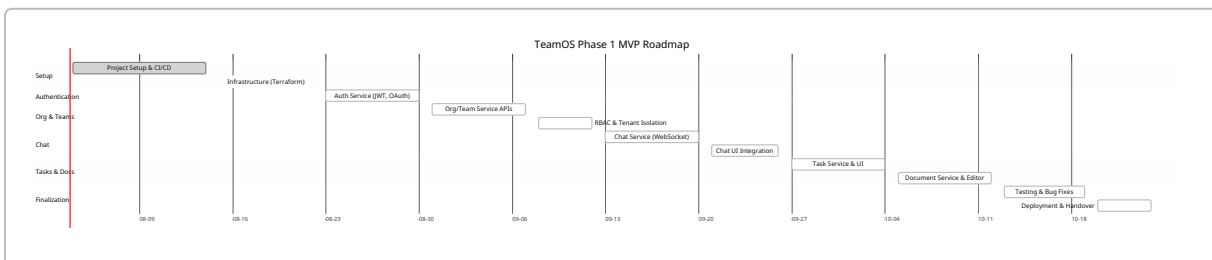
- **Documentation:** Auto-generate API docs (Swagger). Include usage examples in code comments. Provide a clear README.

Roadmap, Milestones, and KPIs

We break Phase 1 into milestones, each with objectives and acceptance criteria:

Milestone	Key Tasks	Acceptance Criteria	KPIs
Setup & Baseline	Set up repo structure, Docker, Terraform, CI pipelines	Repos build successfully; DB and infra provisioned	CI build passing; environment deployment time
Auth Module	Implement User entity, signup/login, JWT tokens, OAuth placeholders	Users can register/login; valid JWT returned	# of successful logins; auth API coverage
Org/Team Module	Org & Team entities, CRUD APIs, RBAC roles	Can create orgs/teams; user roles enforced	Orgs created; RBAC rules tested
Chat Module	WebSocket/STOMP server and client integration	Real-time chat works between users in same team	Message round-trip latency; #messages sent
Task Module	Task CRUD APIs and UI	Tasks can be created/updated by authorized users	Tasks list uptime; # of tasks created
Docs Module	Document CRUD and editor	Documents saved and retrieved correctly	Documents created; content consistency
Final QA & Polish	End-to-end testing, security audit, code cleanup, README	All automated tests pass; demo end-to-end flow works	Test coverage; bug count; demo readiness

Below is a **Gantt chart** (Mermaid) visualizing the timeline (assuming ~8-12 weeks total):



Architecture Trade-offs and Database Models

Microservices vs Modular Monolith: At first, a single modular monolith can simplify development and deployment [25†L68-L72] . Microservices (many small services [25†L58-L64]) offer independent

scalability and fault isolation, but add networking and orchestration complexity. Our phased approach starts monolithic (with clear module boundaries) and extracts services later as needed.

DB Multi-Tenancy Models: AWS outlines three models [11†L22-L26] . In summary:

Model	Description (Pros/Cons)
Silo	Separate database per tenant. High isolation (no cross-tenant risk) [11†L33-L41] , per-tenant tuning, but poor agility and high cost. Complex deployment.
Pool	Shared DB with a <code>tenant_id</code> column. Easier management and cost-effective [11†L51-L59] , but requires strict row-level filtering (some cross-tenant impact).
Bridge	Hybrid: some shared resources, some separate. Balances agility and isolation [11†L67-L75] , but adds complexity.

For Phase 1 we use the **Pool** model for simplicity and speed of development.

Comprehensive Codex Prompt

The following is the **Codex prompt** to generate the TeamOS Phase 1 project. It is self-contained and assumes the repository contains `PRD.md` , `DESIGN.md` , and `TRD.md` (which describe the product requirements and design). Unspecified details (cloud provider, domain) default to AWS and a generic setup. The prompt instructs generating all scaffolding, code, configs, and documentation described above:

```
# TeamOS Phase 1 MVP Implementation
```

```
Implement a TeamOS Phase 1 (MVP) workspace application according to the PRD, Design Spec, and TRD provided in the repository. Use Java Spring Boot (version 3+) for backend services and Next.js (React + TypeScript) for the frontend. Assume AWS-compatible cloud by default (PostgreSQL on RDS, Redis via ElastiCache, Cloudflare R2 as S3, etc.). Provide code, config, and documentation as follows:
```

```
1. Project Structure:
```

```
- Organize the code into a Git-like repo structure. For example:
```

```
```\n\n/backend\n  /auth-service      // Spring Boot (authentication, users)\n  /org-service       // Spring Boot (organizations, teams)\n  /chat-service      // Spring Boot (chat messaging, WebSockets)\n  /task-service       // Spring Boot (tasks)\n  /doc-service        // Spring Boot (documents)\n  /api-gateway        // (Optional) Spring Cloud Gateway or similar\n  Dockerfile          // Common Dockerfile or one per service\n  docker-compose.yml
```

```

k8s/ // Kubernetes manifests for each service
terraform/ // Terraform templates for DB, Redis, R2, etc.
.github/workflows // CI/CD GitHub Actions
README.md
/frontend
 /pages // Next.js pages (login, dashboard, org/team pages, chat,
tasks, docs)
 /components // React UI components (forms, chat window, etc.)
 /lib // API client, auth context
 Dockerfile // Next.js Dockerfile
 .github/workflows
 README.md

```

...

- Include a **top-level README.md** with setup/run instructions.

## **\*\*2. Backend Services:\*\***

- **\*\*Spring Boot Apps:\*\*** Create scaffold Spring Boot applications for each service. Use Spring Web, Spring Data JPA (PostgreSQL), and Spring Security. Configure the main application and application.yml.
- **\*\*Database Schema:\*\*** Define JPA entities and SQL migrations for all tables, including fields and indexes. Sample tables: `users`, `organizations`, `teams`, `messages`, `tasks`, `documents`. Include `tenant\_id` to isolate org data. Provide example `CREATE TABLE` statements and use Flyway or Liquibase.
- **\*\*Auth & Security:\*\*** Implement JWT authentication. Provide `/auth/signup` and `/auth/login` endpoints. Issue JWT access tokens and refresh tokens. In controllers/services, enforce full JWT validation (signature, issuer, audience) and use short expiration (best practices [\[27†L261-L267\]](#)). Include optional OAuth2/OIDC login stubs (e.g. Google). Write Spring Security config to secure endpoints and handle roles (ADMIN, MEMBER).
- **\*\*Organization & Team Management:\*\*** Endpoints under `/api/orgs` to create/list orgs, and `/api/orgs/{orgId}/teams` for team CRUD. Include controllers, services, repositories. Ensure `tenant\_id` filtering. Example endpoint: `POST /api/orgs` with JSON `{ "name": "Acme Corp" }` returns org details.
- **\*\*Chat (WebSockets):\*\*** Configure STOMP over WebSocket (`@EnableWebSocketMessageBroker` in a WebSocketConfig)` [\[17†L259-L263\]](#). Define an endpoint `/ws`. Create a message controller to handle destinations (e.g. `/app/chat.sendMessage`) and broadcast to `/topic/team.{teamId}`. Sample WebSocket message format: JSON with `teamId`, `senderId`, `content`, `timestamp`.
- **\*\*Tasks:\*\*** Endpoints for task management (`/api/tasks`). Implement `Task` entity with fields (id, title, description, assigneeId, teamId, status, etc.). Example: `POST /api/tasks` accepts task JSON and returns created task. Include repository and service.
- **\*\*Documents:\*\*** Endpoints for documents (`/api/docs`). Store document content (rich text or Markdown). Optionally store large content in R2/S3: upload document files via Cloudflare R2 using AWS SDK. Provide example: `POST /api/docs` with `{ "title": "Spec", "content": "# Plan...", "teamId": 1 }`.

- **\*\*API Documentation:\*\*** Use springdoc-openapi to generate OpenAPI/Swagger. Include annotations for sample requests/responses.
- **\*\*Error Handling:\*\*** Add global exception handlers to return proper HTTP error codes and messages.

**\*\*3. Frontend (Next.js):\*\***

- **\*\*App Setup:\*\*** Initialize Next.js app with TypeScript. Set up `pages` for routes.
- **\*\*Authentication UI:\*\*** Build `/login` and `/signup` pages with React components (forms). On submit, call backend `/auth/signup` or `/auth/login`, handle responses (store JWT token, e.g. in HttpOnly cookie or memory). Redirect to a dashboard on success.
- **\*\*Protected Routes:\*\*** Create a mechanism (HOC or middleware) to check JWT before allowing access to protected pages (org/team/dashboard).
- **\*\*Organization/Team UI:\*\*** Pages to view current organization and teams. Allow creating new teams (calls backend), inviting members, etc.
- **\*\*Chat UI:\*\*** A page (e.g. `/teams/[teamId]/chat`) that uses a WebSocket client (e.g. `@stomp/stompjs` or native) to connect to the backend STOMP endpoint. Subscribe to the team topic, display incoming messages, and send messages via the STOMP client.
- **\*\*Task UI:\*\*** Pages and components to list and create tasks for a team. Use API calls to backend. Include forms for creating/updating tasks.
- **\*\*Document Editor:\*\*** A page to create/edit documents. Use a Markdown editor component or rich text editor. On save, send to backend. Show list of docs and allow opening for editing.
- **\*\*Navigation & Layout:\*\*** Include a header with org selector, user profile/logout. Common layout.
- **\*\*Build & Config:\*\*** Next.js `next.config.js`, environment variables for API base URL.
- **\*\*Styling:\*\*** Use CSS framework (e.g. Tailwind or plain CSS) for clean UI.
- **\*\*Frontend Testing:\*\*** Write at least basic tests for login form and one dashboard component using Jest.

**\*\*4. Real-time Chat Implementation:\*\***

- Describe the WebSocket/STOMP setup in detail. For example:
  - In Spring Boot: `@EnableWebSocketMessageBroker`,  
`registry.enableSimpleBroker("/topic")`,  
`registry.setApplicationDestinationPrefixes("/app")`.
  - In Next.js: initialize STOMP client: `const client = new StompJs.Client({ brokerURL: 'ws://localhost:8080/ws' });`, then  
`client.subscribe('/topic/team.{teamId}', (msg) => { ... })`.
- Include **\*\*example message structure\*\*** and how messages flow.

**\*\*5. CI/CD and DevOps Artifacts:\*\***

- **\*\*GitHub Actions:\*\*** Provide workflow YAML examples (as shown above) for building backend and frontend.
- **\*\*Docker & Docker Compose:\*\*** Example `docker-compose.yml` to run Postgres, Redis, and all services for local dev.

- **Kubernetes Manifests:** Example `deployment.yaml` and `service.yaml` for one or two services and the frontend.
- **Terraform:** Example `main.tf` for AWS RDS instance, ElastiCache Redis, and S3 bucket (for R2). Use AWS Provider.

#### **6. Tables and Diagrams:**

- **Architecture Trade-off Table:** Include a markdown table (like above) comparing Monolith vs Microservices trade-offs, citing definitions where useful [25†L58-L64] [25†L68-L72] .
- **Database Partitioning Table:** Include a table summarizing Silo vs Pool vs Bridge models (as above), citing AWS guidance [11†L33-L41] [11†L51-L59] .
- **Timeline Gantt Chart:** Provide a Mermaid Gantt chart showing milestones (like the one above).

#### **7. Deliverables:**

The generated output should be a **fully scaffolded Git repository** with all code, configs, and a README. Code should contain comments/docstrings. The README should explain how to set up (e.g. running Terraform, building Docker images, running the dev server). All of the above should be contained in the codex-generated prompt response, with the expectation that Codex will output code/files accordingly.